

■ ■ Azure Cloud

Lab Workbook — Questions & Solutions

Portal • Azure CLI • Every Major Topic

15 Lab Modules • 120+ Exam-Style Questions • Full Step-by-Step Solutions

Resource Groups • VNets • VMs • App Service • Blob Storage • SQL

RBAC • Entra ID • Key Vault • Monitor • App Insights • DevOps • Pipelines

■ Resource Groups	■ VNets & Subnets	■ Virtual Machines	■ App Service
■ Blob Storage	■ ■ SQL Database	■ Entra ID & RBAC	■ Key Vault
■ App Insights	■ Azure Monitor	■ Azure DevOps	■ ■ YAML Pipelines

Table of Contents

PART 1 — FOUNDATIONS

- Lab 1: Resource Groups & Subscriptions
- Lab 2: Azure CLI & Cloud Shell

PART 2 — NETWORKING

- Lab 3: Virtual Networks & Subnets
- Lab 4: Network Security Groups (NSGs)

PART 3 — COMPUTE

- Lab 5: Azure Virtual Machines
- Lab 6: Azure App Service

PART 4 — STORAGE & DATA

- Lab 7: Azure Blob Storage
- Lab 8: Azure SQL Server & Database

PART 5 — IDENTITY & SECURITY

- Lab 9: Azure Entra ID (Azure AD)
- Lab 10: Azure RBAC
- Lab 11: Service Principals & Key Vault

PART 6 — MONITORING

- Lab 12: Application Insights
- Lab 13: Azure Monitor & Alerts

PART 7 — DEVOPS

- Lab 14: Azure Repos & Boards
- Lab 15: Azure YAML Pipelines

PART 1

Foundations

Resource Groups, Subscriptions & Azure CLI

Lab 1: Resource Groups & Subscriptions

Lab 2: Azure CLI & Cloud Shell

LAB 1

Resource Groups & Subscriptions

■ 45 min

Difficulty: ★■■

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create and manage resource groups via Portal and CLI
- Apply tags to resource groups
- Add resource locks to prevent deletion
- Move resources between resource groups
- Understand subscription scoping

Q1

Create a resource group named 'rg-lab-dev' in the East US region with tags Environment=Dev and Owner=Student.

Easy

Scenario: You are starting a new dev project and need a container for all resources.

■ Portal Solution

- 1 Open Resource Groups**
In the Azure Portal, type 'Resource groups' in the top search bar. Click the service.
- 2 Create new**
Click '+ Create' button at top-left.
- 3 Fill Basics**
Subscription: select your subscription. Resource group: type rg-lab-dev. Region: (US) East US.
- 4 Add Tags**
Click 'Next: Tags'. Add row 1: Name=Environment, Value=Dev. Row 2: Name=Owner, Value=Student.
- 5 Review + Create**
Click 'Review + create'. Verify all values. Click 'Create'. Wait ~10 seconds.
- 6 Verify**
Resource group appears in the list. Click it and verify region and tags in the Overview pane.

■ Azure CLI Solution

```
# Step 1: Login (skip if already logged in)
az login

# Step 2: Create the resource group with tags
az group create \
  --name rg-lab-dev \
  --location eastus \
  --tags Environment=Dev Owner=Student

# Step 3: Verify it was created
az group show --name rg-lab-dev --output table

# Step 4: Verify tags
az group show --name rg-lab-dev --query tags
```

```
# Expected output:
# {
#   "Environment": "Dev",
#   "Owner": "Student"
# }
```

■ **Explanation:** Resource groups are free containers. Tags are key-value metadata pairs used for billing, filtering, and automation. Always tag your resources!

Q2

Add a Delete lock to 'rg-lab-dev' so it cannot be accidentally deleted.

Medium

Scenario: Production resource groups must be protected from accidental deletion.

■ Portal Solution

1	Open resource group Navigate to rg-lab-dev in the portal.
2	Locks In the left menu, scroll down and click 'Locks' under Settings.
3	Add lock Click '+ Add'. Lock name: lock-nodelete. Lock type: Delete. Notes: Prevents accidental deletion. Click 'OK'.
4	Verify The lock appears in the list. Try clicking 'Delete' on the resource group — you'll get an error!

■ Azure CLI Solution

```
# Add a delete lock to the resource group
az lock create \
  --name lock-nodelete \
  --resource-group rg-lab-dev \
  --lock-type CanNotDelete \
  --notes 'Prevents accidental deletion of dev RG'
# List locks on the resource group
az lock list --resource-group rg-lab-dev --output table
# Try to delete RG — should FAIL with lock error
az group delete --name rg-lab-dev --yes
# Expected: AuthorizationFailed or CanNotDelete error
# Remove the lock first before deleting
az lock delete \
  --name lock-nodelete \
  --resource-group rg-lab-dev
```

■ **Explanation:** Locks are CanNotDelete (allow read + modify, deny delete) or ReadOnly (allow read only). They override RBAC — even Owners can't delete a locked resource without removing the lock first.

Q3

Create three resource groups for different environments: rg-myapp-dev, rg-myapp-staging, rg-myapp-prod. Each should have appropriate environment tags.

Medium

Scenario: Your team uses separate resource groups per environment for cost tracking and access control.

■ Portal Solution

1	Repeat creation 3 times Create each resource group via Portal → Resource groups → + Create with appropriate tag values.
2	Dev RG Name: rg-myapp-dev Tags: Environment=Dev, CostCenter=Engineering
3	Staging RG Name: rg-myapp-staging Tags: Environment=Staging, CostCenter=Engineering
4	Prod RG Name: rg-myapp-prod Tags: Environment=Production, CostCenter=Engineering
5	Add lock to prod Navigate to rg-myapp-prod → Locks → Add a CanNotDelete lock.

■ Azure CLI Solution

```
#!/bin/bash
# Create all 3 environment resource groups in one script
LOCATION='eastus'
declare -A ENVS
ENVS[dev]='Dev'
ENVS[staging]='Staging'
ENVS[prod]='Production'
for ENV in dev staging prod; do
  RG_NAME="rg-myapp-${ENV}"
  ENV_VALUE="${ENVS[${ENV}]}"
  az group create \
    --name $RG_NAME \
    --location $LOCATION \
    --tags Environment=$ENV_VALUE CostCenter=Engineering
  echo "Created: $RG_NAME"
done
# Add delete lock to production only
az lock create \
  --name lock-prod-nodelete \
  --resource-group rg-myapp-prod \
  --lock-type CanNotDelete
# List all resource groups with tag filter
az group list --query "[?tags.CostCenter=='Engineering']" --output table
```

■ *Explanation:* Scripting resource group creation ensures consistency. Using tags consistently enables cost analysis by environment in Azure Cost Management.

Q4 List all resource groups in your subscription and export the result as a JSON file.

Easy

Scenario: You need to audit all resource groups and their metadata for a compliance report.

■ Portal Solution

- Resource groups list**
Go to Resource groups. The list shows all groups with location and subscription.

2	Export Click 'Export to CSV' at the top to download a spreadsheet of all resource groups.
3	Use Azure Resource Graph (advanced) Go to 'Resource Graph Explorer' in portal → run: ResourceContainers where type =~ 'microsoft.resources/subscriptions/resourcegroups'

■ Azure CLI Solution

```
# List all resource groups (table format)
az group list --output table

# List as JSON and save to file
az group list --output json > resource_groups.json

# Pretty print the JSON
az group list --output jsonc

# Filter: only show East US resource groups
az group list \
--query "[?location=='eastus']" \
--output table

# Count total resource groups
az group list --query 'length(@)'

# Show only name and location
az group list \
--query '[].{Name:name, Location:location, Tags:tags}' \
--output table
```

■ *Explanation:* The --query flag uses JMESPath syntax to filter/transform JSON output. --output can be json, jsonc, table, tsv, yaml, or none.

Q5

Delete rg-lab-dev (and all its resources) after removing any existing locks.

Easy

Scenario: The dev project is complete. Clean up all resources to save costs.

■ Portal Solution

1	Remove lock first Go to rg-lab-dev → Locks → click the trash icon next to lock-nodelete.
2	Delete RG Go back to Overview. Click 'Delete resource group' at the top.
3	Confirm Type the resource group name 'rg-lab-dev' in the confirmation box. Click 'Delete'.
4	Wait Deletion may take 2-5 minutes. A notification confirms when done.

■ Azure CLI Solution

```
# Step 1: Remove lock (if it exists)
az lock delete \
--name lock-nodelete \
--resource-group rg-lab-dev 2>/dev/null || echo 'No lock found'

# Step 2: Delete the resource group and ALL contents
az group delete \
--name rg-lab-dev \
```

```
--yes \  
--no-wait  
# --yes: skip confirmation prompt  
# --no-wait: don't block terminal while deleting  
# Step 3: Confirm it's gone (may take a moment)  
az group show --name rg-lab-dev 2>&1 | grep -i 'not found' && echo 'Deleted!' || echo 'Still deleting...'
```

■ *Explanation:* az group delete with --yes --no-wait is the fastest cleanup method. It deletes ALL resources inside. This is why resource groups are so powerful — one command cleans up everything!

LAB 2

Azure CLI & Cloud Shell

■ 30 min

Difficulty: ★■

■ Portal + ■ Azure CLI

■ Learning Objectives

- Set up and use Azure Cloud Shell
- Switch between subscriptions
- Use JMESPath queries to filter output
- Use Azure CLI interactive mode
- Script multiple CLI commands together

Q1

Open Azure Cloud Shell and verify that Azure CLI is installed and working. Display the current subscription details.

Easy

Scenario: This is your first time using Cloud Shell.

■ Portal Solution

1

Open Cloud Shell

In the Azure Portal, click the '>_' icon in the top navigation bar (next to the search bar).

2

First launch

Click 'Bash'. Click 'Create storage' (creates a small file share, < \$1/month). Wait for it to initialize.

3

Verify CLI

Type: `az --version` and press Enter. You'll see the CLI version and all installed extensions.

4

View subscription

Type: `az account show` and press Enter. Shows your current subscription name, ID, and tenant.

■ Azure CLI Solution

```
# Check CLI version
az --version

# Show current account / subscription
az account show

# Show in a cleaner format
az account show --query '{Name:name, ID:id, State:state}' --output table

# List ALL subscriptions you have access to
az account list --output table

# Switch to a different subscription
az account set --subscription 'My Other Subscription'

# Confirm the switch
az account show --query name -o tsv
```

■ *Explanation:* Cloud Shell persists your files in an Azure Storage file share. It has Azure CLI, PowerShell, kubectl, Terraform, and many other tools pre-installed — no setup needed!

Q2

Use Azure CLI to find all resources in your subscription that are tagged with 'Environment=Dev' across ALL resource groups.

Medium

Scenario: You need to inventory all dev resources across the entire subscription for a cost report.

■ Portal Solution

1	Resource Graph Explorer Search 'Resource Graph Explorer' in the portal.
2	Run KQL query Paste this query and click 'Run query':
3	Query Resources where tags['Environment'] == 'Dev' project name, type, resourceGroup, location
4	Export Click 'Download as CSV' to save the results.

■ Azure CLI Solution

```
# Method 1: Using az resource list with tag filter
az resource list \
--tag Environment=Dev \
--output table

# Method 2: Query with JMESPath
az resource list \
--tag Environment=Dev \
--query '[].{Name:name, Type:type, RG:resourceGroup}' \
--output table

# Method 3: Using az graph (if extension installed)
az extension add --name resource-graph
az graph query \
-q "Resources | where tags['Environment'] == 'Dev' | project name, type, resourceGroup"

# Count of dev resources
az resource list --tag Environment=Dev --query 'length(@)'
```

Q3

Use the Azure CLI interactive mode to explore VM commands, then create a resource group using interactive mode.

Medium

Scenario: You want to discover CLI commands interactively with auto-complete.

■ Azure CLI Solution

```
# Install interactive mode extension
az extension add --name interactive

# Enter interactive mode
az interactive

# Inside interactive mode, type:
# >> vm (press Tab to see VM sub-commands)
# >> vm create (shows all parameters with hints)
# >> group create (to create resource group)

# Exit interactive mode
# Press Ctrl+D or type 'exit'

# Without interactive mode, use --help
az vm --help
az vm create --help
```

```
# Use 'az find' (AI-powered search)
az find 'create virtual machine ubuntu'
```

Hard

Scenario: You need to automate resource creation for a repeatable lab setup.

■ Azure CLI Solution

[illegible]

■ *Explanation:* set -e makes the script exit immediately if any command fails. This is important for automation scripts so errors don't get silently ignored.

PART 2

Networking

VNets, Subnets, NSGs & Traffic Control

Lab 3: Virtual Networks & Subnets

Lab 4: Network Security Groups

LAB 3

Virtual Networks & Subnets

■ 45 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create a VNet with a custom address space
- Add multiple subnets for different tiers
- Configure VNet DNS settings
- Set up VNet peering between two VNets
- Verify connectivity between peered VNets

Q1

Create a VNet named 'vnet-lab-prod' with address space 10.10.0.0/16 in East US. Add three subnets: subnet-frontend (10.10.1.0/24), subnet-backend (10.10.2.0/24), and subnet-database (10.10.3.0/24).

Medium

Scenario: You are setting up the network for a 3-tier web application.

■ Portal Solution

1

Search Virtual Networks

Search 'Virtual networks' in the portal. Click '+ Create'.

2

Basics tab

Subscription: yours. Resource group: rg-myapp-dev (create if needed). Name: vnet-lab-prod. Region: East US.

3

IP Addresses tab

IPv4 address space: 10.10.0.0/16. Delete the default subnet. Click '+ Add subnet' three times:

4

Add frontend subnet

Name: subnet-frontend. Starting address: 10.10.1.0. Size: /24. Click Add.

5

Add backend subnet

Name: subnet-backend. Starting address: 10.10.2.0. Size: /24. Click Add.

6

Add database subnet

Name: subnet-database. Starting address: 10.10.3.0. Size: /24. Click Add.

7

Review + Create

Click 'Review + create' then 'Create'. Verify all 3 subnets appear in Subnets tab.

■ Azure CLI Solution

```
# Create resource group first
az group create --name rg-myapp-dev --location eastus

# Create the VNet
az network vnet create \
  --resource-group rg-myapp-dev \
  --name vnet-lab-prod \
  --address-prefix 10.10.0.0/16 \
```

```
--location eastus
# Add all 3 subnets
declare -A SUBNETS=(
[subnet-frontend]='10.10.1.0/24'
[subnet-backend]='10.10.2.0/24'
[subnet-database]='10.10.3.0/24'
)
for SUBNET in "${!SUBNETS[@]}"; do
az network vnet subnet create \
--resource-group rg-myapp-dev \
--vnet-name vnet-lab-prod \
--name $SUBNET \
--address-prefix ${SUBNETS[$SUBNET]}
echo "Created subnet: $SUBNET (${SUBNETS[$SUBNET]})"
done
# Verify all subnets
az network vnet subnet list \
--resource-group rg-myapp-dev \
--vnet-name vnet-lab-prod \
--output table
```

Q2

Create a second VNet 'vnet-lab-mgmt' (10.20.0.0/16) and peer it with 'vnet-lab-prod' so both can communicate.

Hard

Scenario: Your management tools in vnet-lab-mgmt need to connect to app servers in vnet-lab-prod.

■ Portal Solution

1

Create vnet-lab-mgmt

Repeat VNet creation: name vnet-lab-mgmt, space 10.20.0.0/16, add subnet-mgmt (10.20.1.0/24).

2

Open vnet-lab-prod

Navigate to vnet-lab-prod.

3

Peerings

In left menu, click 'Peerings'. Click '+ Add'.

4

Configure peering

This VNet peering name: peer-prod-to-mgmt. Remote VNet: vnet-lab-mgmt. Remote peering name: peer-mgmt-to-prod. Allow traffic: Yes (both directions). Click 'Add'.

5

Verify

Both peerings show 'Connected' status. Also check vnet-lab-mgmt → Peerings — you'll see the reverse peering automatically created.

■ Azure CLI Solution

```
# Create management VNet
az network vnet create \
--resource-group rg-myapp-dev \
--name vnet-lab-mgmt \
--address-prefix 10.20.0.0/16
az network vnet subnet create \
--resource-group rg-myapp-dev \
--vnet-name vnet-lab-mgmt \
--name subnet-mgmt \
```

```
--address-prefix 10.20.1.0/24
# Get resource IDs for peering
PROD_ID=$(az network vnet show \
--resource-group rg-myapp-dev \
--name vnet-lab-prod --query id -o tsv)
MGMT_ID=$(az network vnet show \
--resource-group rg-myapp-dev \
--name vnet-lab-mgmt --query id -o tsv)
# Peer prod -> mgmt
az network vnet peering create \
--resource-group rg-myapp-dev \
--name peer-prod-to-mgmt \
--vnet-name vnet-lab-prod \
--remote-vnet $MGMT_ID \
--allow-vnet-access \
--allow-forwarded-traffic
# Peer mgmt -> prod (MUST be created in both directions)
az network vnet peering create \
--resource-group rg-myapp-dev \
--name peer-mgmt-to-prod \
--vnet-name vnet-lab-mgmt \
--remote-vnet $PROD_ID \
--allow-vnet-access \
--allow-forwarded-traffic
# Verify peering state
az network vnet peering list \
--resource-group rg-myapp-dev \
--vnet-name vnet-lab-prod \
--query '[].{Name:name, State:peeringState}' \
--output table
```

■ **Explanation:** VNet peering is non-transitive: if VNet-A peers with VNet-B and VNet-B peers with VNet-C, VNet-A cannot talk to VNet-C through B. You must peer A-C directly, or use Azure Virtual WAN / Hub-and-Spoke with NVA.

Q3

Show the effective routes for a VM's network interface card (NIC) to understand how traffic flows.

Hard

Scenario: You need to troubleshoot why a VM cannot reach the internet.

■ Portal Solution

1 Go to the NIC

Navigate to your VM → Networking → Click the NIC name link.

2 Effective routes

In NIC's left menu, click 'Effective routes' under Help.

3 Analyse

Look for a 0.0.0.0/0 (default) route. If source is 'Default', traffic goes to internet. If source is 'Virtual network gateway' or 'None', it may be blocked.

■ Azure CLI Solution

```
# First, find the NIC name of your VM
az vm show \
--resource-group rg-myapp-dev \
```

```
--name vm-test-01 \  
--query 'networkProfile.networkInterfaces[].id' \  
--output tsv  
  
# Get effective routes for the NIC  
az network nic show-effective-route-table \  
--resource-group rg-myapp-dev \  
--name vm-test-01-nic \  
--output table  
  
# Get effective security rules (NSG rules applied)  
az network nic list-effective-nsg \  
--resource-group rg-myapp-dev \  
--name vm-test-01-nic
```

LAB 4

Network Security Groups (NSGs)

■ 40 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create NSGs with custom inbound and outbound rules
- Associate NSGs with subnets and NICs
- Test and troubleshoot NSG rules
- Understand NSG rule priority
- Use NSG flow logs for traffic analysis

Q1

Create an NSG named 'nsg-web' that allows HTTP (80), HTTPS (443) inbound from internet, allows SSH (22) ONLY from IP 10.0.0.0/8, and denies all other inbound traffic.

Medium

Scenario: Secure the web-tier subnet so only web traffic reaches it.

■ Portal Solution

1

Create NSG

Search 'Network security groups' → '+ Create'. Name: nsg-web. Resource group: rg-myapp-dev.

2

Add HTTP rule

Go to nsg-web → Inbound security rules → '+ Add'. Source: Any. Source port: *. Dest: Any. Dest port: 80. Protocol: TCP. Action: Allow. Priority: 100. Name: Allow-HTTP.

3

Add HTTPS rule

Same as above but Dest port: 443. Priority: 110. Name: Allow-HTTPS.

4

Add SSH rule

Source: IP Addresses. Source IP: 10.0.0.0/8. Dest port: 22. Protocol: TCP. Priority: 120. Name: Allow-SSH-Internal.

5

Deny-All rule

Source: Any. Port: *. Protocol: Any. Action: DENY. Priority: 4000. Name: Deny-All-Inbound. (Note: default rules already deny, this makes it explicit)

■ Azure CLI Solution

```
# Create NSG
az network nsg create \
  --resource-group rg-myapp-dev \
  --name nsg-web
# Allow HTTP (port 80)
az network nsg rule create \
  --resource-group rg-myapp-dev \
  --nsg-name nsg-web \
  --name Allow-HTTP \
  --priority 100 \
  --direction Inbound \
```

```
--source-address-prefixes Internet \
--source-port-ranges '*' \
--destination-address-prefixes '*' \
--destination-port-ranges 80 \
--protocol Tcp \
--access Allow
# Allow HTTPS (port 443)
az network nsg rule create \
--resource-group rg-myapp-dev \
--nsg-name nsg-web \
--name Allow-HTTPS \
--priority 110 \
--direction Inbound \
--source-address-prefixes Internet \
--destination-port-ranges 443 \
--protocol Tcp \
--access Allow
# Allow SSH only from internal 10.x.x.x network
az network nsg rule create \
--resource-group rg-myapp-dev \
--nsg-name nsg-web \
--name Allow-SSH-Internal \
--priority 120 \
--direction Inbound \
--source-address-prefixes '10.0.0.0/8' \
--destination-port-ranges 22 \
--protocol Tcp \
--access Allow
# Explicit Deny-All (optional – default rule already denies)
az network nsg rule create \
--resource-group rg-myapp-dev \
--nsg-name nsg-web \
--name Deny-All-Inbound \
--priority 4000 \
--direction Inbound \
--source-address-prefixes '*' \
--destination-port-ranges '*' \
--protocol '*' \
--access Deny
# List all rules
az network nsg rule list \
--resource-group rg-myapp-dev \
--nsg-name nsg-web \
--output table
# Associate NSG with a subnet
az network vnet subnet update \
--resource-group rg-myapp-dev \
--vnet-name vnet-lab-prod \
--name subnet-frontend \
--network-security-group nsg-web
```

■ **Explanation:** NSG rules are evaluated lowest priority number first (100 before 200). Once a match is found, evaluation stops. Default Azure rules at priorities 65000-65500 allow VNet traffic and deny all else.

Q2

Use the NSG IP Flow Verify tool to test if traffic from IP 203.0.113.10 on port 80 would be allowed to reach a VM in the web subnet.

Medium

Scenario: You need to verify NSG rules are working before going live.

■ Portal Solution

1	Open Network Watcher Search 'Network Watcher' in the portal.
2	IP flow verify In left menu under 'Network diagnostic tools', click 'IP flow verify'.
3	Configure test Subscription: yours. Resource group: rg-myapp-dev. VM: your web VM. NIC: select the NIC. Direction: Inbound. Protocol: TCP. Local IP: VM's private IP. Local port: 80. Remote IP: 203.0.113.10. Remote port: any (e.g., 52000).
4	Run Click 'Check'. Result will say 'Access allowed' or 'Access denied' and which rule matched.

■ Azure CLI Solution

```
# IP Flow Verify via CLI
az network watcher test-ip-flow \
--resource-group rg-myapp-dev \
--vm vm-web-01 \
--direction Inbound \
--protocol TCP \
--local-ip 10.10.1.4 \
--local-port 80 \
--remote-ip 203.0.113.10 \
--remote-port 52000

# Expected output:
# {
#   'access': 'Allow',
#   'ruleName': 'Allow-HTTP'
# }
```


PART 3

Compute & Hosting

Virtual Machines & App Service

Lab 5: Azure Virtual Machines

Lab 6: Azure App Service

LAB 5

Azure Virtual Machines

■ 60 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create a Linux VM with SSH key authentication
- Connect via SSH and install a web server
- Attach and format a data disk
- Create a VM from a custom image / snapshot
- Configure VM auto-shutdown
- Resize a VM to a different SKU

Q1

Create a Standard_B1s Ubuntu 22.04 VM named 'vm-web-lab' in rg-myapp-dev. Enable SSH, open port 80, and configure auto-shutdown at 22:00 UTC.

Medium

Scenario: Set up a basic web server VM for a dev environment.

■ Portal Solution

1	Create VM Search 'Virtual machines' → '+ Create' → 'Azure virtual machine'.
2	Basics RG: rg-myapp-dev. Name: vm-web-lab. Region: East US. Image: Ubuntu Server 22.04 LTS. Size: Standard_B1s. Auth: SSH public key. Username: azureuser. Key: Generate new → download the .pem file.
3	Inbound ports Allow SSH (22) and HTTP (80).
4	Disks OS disk: Standard SSD. No data disk yet.
5	Management tab Enable auto-shutdown: ON. Shutdown time: 10:00 PM. Timezone: your timezone. Email: your email.
6	Review + Create Review estimated cost. Click 'Create'. Download key pair when prompted.
7	Verify After deployment, go to the VM → copy the Public IP. SSH into it.

■ Azure CLI Solution

```
# Create VM with SSH keys auto-generated
az vm create \
  --resource-group rg-myapp-dev \
  --name vm-web-lab \
  --image Ubuntu2204 \
  --size Standard_B1s \
```

```
--admin-username azureuser \
--generate-ssh-keys \
--public-ip-sku Standard \
--output json
# Open port 80 for HTTP
az vm open-port \
--resource-group rg-myapp-dev \
--name vm-web-lab \
--port 80
# Configure auto-shutdown at 22:00 UTC
az vm auto-shutdown \
--resource-group rg-myapp-dev \
--name vm-web-lab \
--time 2200 \
--email 'your@email.com'
# Get public IP
az vm list-ip-addresses \
--resource-group rg-myapp-dev \
--name vm-web-lab \
--query '[0].virtualMachine.network.publicIpAddresses[0].ipAddress' \
--output tsv
```

Q2

SSH into 'vm-web-lab', install nginx, create a custom HTML page, and verify it's accessible via the public IP.

Medium

Scenario: Deploy a simple website to your VM.

■ Azure CLI Solution

```
# Get the public IP first
PUBLIC_IP=$(az vm list-ip-addresses \
--resource-group rg-myapp-dev \
--name vm-web-lab \
--query '[0].virtualMachine.network.publicIpAddresses[0].ipAddress' \
--output tsv)
echo "Connecting to: $PUBLIC_IP"
# SSH connect (key auto-generated is at ~/.ssh/id_rsa)
ssh -o StrictHostKeyChecking=no azureuser@$PUBLIC_IP
# ■■■ COMMANDS TO RUN INSIDE THE VM ■■■■■■■■■■
# Update package lists
sudo apt update -y
# Install nginx
sudo apt install nginx -y
# Create a custom homepage
sudo tee /var/www/html/index.html << 'EOF'
<!DOCTYPE html>
<html>
<head><title>Azure Lab VM</title></head>
<body style='font-family:sans-serif; text-align:center; margin-top:100px'>
<h1>Hello from Azure VM!</h1>
<p>Hosted on Ubuntu 22.04 in Azure</p>
<p>Lab 5 Complete!</p>
</body>
</html>
EOF
# Start and enable nginx
```

Q3

Hard

■ Portal Solution

1

Navigate to vm-web-lab → Disks in the left menu.

2

Click '+ Create and attach a new disk'. Name: vm-web-lab-datadisk01. Type: Standard SSD. Size: 32 GiB. Click Save.

3

SSH into the VM. Find the disk (usually `/dev/sdc`). Format and mount it.

Page 26

```
# Should show 32GB available
```

Q4

Stop and deallocate 'vm-web-lab', then resize it from Standard_B1s to Standard_B2s.

Medium

Scenario: The VM needs more CPU and memory for upcoming load tests.

■ Portal Solution

1

Stop VM

Navigate to vm-web-lab → click 'Stop'. Wait for status to show 'Stopped (deallocated)'.

2

Resize

In the left menu click 'Size'. Search for 'B2s'. Select 'Standard_B2s'. Click 'Resize'.

3

Restart

After resize completes, click 'Start' to boot the VM.

4

Verify

Go to Overview. The size should now show Standard_B2s.

■ Azure CLI Solution

```
# Stop and deallocate the VM
az vm deallocate \
--resource-group rg-myapp-dev \
--name vm-web-lab

# Check available sizes in the region
az vm list-vm-resize-options \
--resource-group rg-myapp-dev \
--name vm-web-lab \
--query '[] .name' \
--output tsv | grep -i b2

# Resize the VM
az vm resize \
--resource-group rg-myapp-dev \
--name vm-web-lab \
--size Standard_B2s

# Start the VM
az vm start \
--resource-group rg-myapp-dev \
--name vm-web-lab

# Verify the new size
az vm show \
--resource-group rg-myapp-dev \
--name vm-web-lab \
--query 'hardwareProfile.vmSize' \
--output tsv
```

LAB 6

Azure App Service

■ 45 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create an App Service Plan and Web App
- Deploy a Node.js app from GitHub
- Configure environment variables and connection strings
- Set up deployment slots and perform a slot swap
- Configure custom domain and auto-scaling

Q1

Create a Free-tier App Service Plan and deploy a Node.js 'Hello World' web app named 'webapp-lab-[yourname]'.

Easy

Scenario: Your first cloud-hosted web application!

■ Portal Solution

- 1 App Services**
Search 'App Services' → '+ Create' → 'Web App'.
- 2 Basics**
RG: rg-myapp-dev. Name: webapp-lab-yourname (globally unique!). Publish: Code. Runtime: Node 18 LTS. OS: Linux. Region: East US.
- 3 App Service Plan**
Click 'Create new'. Name: asp-lab-free. SKU: Free F1.
- 4 Review + Create**
Create and wait. Click URL to see default page: webapp-lab-yourname.azurewebsites.net
- 5 Deploy code**
Go to App Service → Deployment Center → Source: GitHub → authorize → choose your repo.

■ Azure CLI Solution

```
# Create App Service Plan (Free tier)
az appservice plan create \
--name asp-lab-free \
--resource-group rg-myapp-dev \
--sku FREE \
--is-linux

# Create Web App
YOUR_NAME='studentXXX' # change this!
az webapp create \
--resource-group rg-myapp-dev \
--plan asp-lab-free \
--name webapp-lab-$YOUR_NAME \
--runtime 'NODE:18-lts'

# Create a simple Node.js app locally
```

```
mkdir myapp && cd myapp
cat > index.js << 'EOF'
const http = require('http');
const port = process.env.PORT || 3000;
http.createServer((req, res) => {
res.writeHead(200, {'Content-Type': 'text/html'});
res.end('<h1>Hello from Azure App Service!</h1><p>Lab 6 Complete!</p>');
}).listen(port, () => console.log(`Server on port ${port}`));
EOF
cat > package.json << 'EOF'
{"name": "myapp", "version": "1.0.0", "main": "index.js", "scripts": {"start": "node index.js"}}
EOF
# Deploy using az webapp up (creates zip and deploys)
az webapp up \
--resource-group rg-myapp-dev \
--name webapp-lab-$YOUR_NAME \
--runtime 'NODE:18-lts'
# Open in browser
az webapp browse \
--resource-group rg-myapp-dev \
--name webapp-lab-$YOUR_NAME
```

Q2

Add three app settings (environment variables) to your web app: DB_HOST, DB_PORT=5432, and APP_ENV=development. Then read them inside the app.

Easy

Scenario: Your app needs to read database connection details from environment variables, not hardcoded in code.

■ Portal Solution

- 1 App Service → Configuration**
Navigate to webapp-lab-yourname → Configuration (under Settings).
- 2 Application settings tab**
Click '+ New application setting'.
- 3 Add settings**
Add: DB_HOST = mydbserver.postgres.database.azure.com. Add: DB_PORT = 5432. Add: APP_ENV = development.
- 4 Save**
Click 'Save' at the top. The app restarts automatically.
- 5 Read in app**
In Node.js: process.env.DB_HOST, process.env.DB_PORT, process.env.APP_ENV

■ Azure CLI Solution

```
# Set multiple app settings at once
az webapp config appsettings set \
--resource-group rg-myapp-dev \
--name webapp-lab-$YOUR_NAME \
--settings \
DB_HOST='mydbserver.postgres.database.azure.com' \
DB_PORT='5432' \
APP_ENV='development'
# List all app settings
```

```
az webapp config appsettings list \
--resource-group rg-myapp-dev \
--name webapp-lab-$YOUR_NAME \
--output table
# Delete a setting
az webapp config appsettings delete \
--resource-group rg-myapp-dev \
--name webapp-lab-$YOUR_NAME \
--setting-names DB_HOST
# Access in Node.js code:
# const dbHost = process.env.DB_HOST;
# const dbPort = process.env.DB_PORT;
# const env = process.env.APP_ENV;
```

Q3

Create a 'staging' deployment slot, deploy new code to staging, verify it, then swap staging to production with zero downtime.

Hard

Scenario: You need to test new code in a staging environment before it hits production users.

■ Portal Solution

1	Upgrade plan Deployment slots require Standard tier. Go to App Service → Scale up (App Service plan) → S1.
2	Create slot App Service → Deployment slots → '+ Add Slot'. Name: staging. Clone settings from: production. Click 'Add'.
3	Deploy to staging Click on the staging slot → Deployment Center → deploy new code to staging slot.
4	Test staging URL Staging URL: webapp-lab-yourname-staging.azurewebsites.net — verify new code is there.
5	Swap Go back to main App Service → Deployment slots → 'Swap'. Source: staging, Target: production. Click Swap.
6	Verify Your main URL now shows the new code. Staging shows the old code (swapped back for rollback).

■ Azure CLI Solution

```
# Upgrade plan to Standard (needed for slots)
az appservice plan update \
--name asp-lab-free \
--resource-group rg-myapp-dev \
--sku S1
# Create staging slot
az webapp deployment slot create \
--resource-group rg-myapp-dev \
--name webapp-lab-$YOUR_NAME \
--slot staging \
--configuration-source webapp-lab-$YOUR_NAME
# Deploy new code to staging slot (using zip deploy)
# (update index.js with 'version 2' message first)
```

```
zip -r app.zip index.js package.json
az webapp deployment source config-zip \
--resource-group rg-myapp-dev \
--name webapp-lab-${YOUR_NAME} \
--slot staging \
--src app.zip
# Test staging URL
echo "Staging: https://webapp-lab-${YOUR_NAME}-staging.azurewebsites.net"
# Swap staging to production
az webapp deployment slot swap \
--resource-group rg-myapp-dev \
--name webapp-lab-${YOUR_NAME} \
--slot staging \
--target-slot production
# Test production URL (should now show v2)
curl https://webapp-lab-${YOUR_NAME}.azurewebsites.net
```

■ *Explanation:* Slot swap is instant and zero-downtime — Azure routes traffic to the new slot atomically. If something breaks, swap back to restore the old version in seconds!

PART 4

Storage & Databases

Blob Storage & Azure SQL

Lab 7: Azure Blob Storage

Lab 8: Azure SQL Database

LAB 7

Azure Blob Storage

■ 40 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create storage accounts with different redundancy levels
- Upload and download blobs via Portal and CLI
- Configure blob access levels (public vs private)
- Generate SAS tokens for time-limited access
- Host a static website from Blob Storage
- Configure lifecycle management policies

Q1

Create a storage account named 'stlab[yourname]' with LRS redundancy and Hot access tier. Create two containers: 'public-images' (public blob access) and 'private-docs' (private access).

Easy

Scenario: Your app stores public images and private documents in separate containers.

■ Portal Solution

1

Create storage account

Search 'Storage accounts' → '+ Create'. RG: rg-myapp-dev. Name: stlabyourname (3-24 lowercase alphanumeric). Region: East US. Performance: Standard. Redundancy: LRS.

2

Advanced tab

Enable 'Allow enabling anonymous access on individual containers'.

3

Create

Click 'Review + create' then 'Create'.

4

Create public container

Go to storage account → Containers → '+ Container'. Name: public-images. Access level: Blob (anonymous read for blobs only).

5

Create private container

Click '+ Container'. Name: private-docs. Access level: Private (no anonymous access).

■ Azure CLI Solution

```
STORAGE='stlab$(echo $USER | head -c4)$(date +%s | tail -c4)'\necho "Storage account name: $STORAGE"\n# Create storage account\naz storage account create \\\n--name $STORAGE \\\n--resource-group rg-myapp-dev \\\n--location eastus \\\n--sku Standard_LRS \\\n--kind StorageV2 \\\n--access-tier Hot \\\n--allow-blob-public-access true
```

```
# Get account key for auth
ACCOUNT_KEY=$(az storage account keys list \
--account-name $STORAGE \
--resource-group rg-myapp-dev \
--query '[0].value' -o tsv)

# Create public container
az storage container create \
--account-name $STORAGE \
--account-key $ACCOUNT_KEY \
--name public-images \
--public-access blob

# Create private container
az storage container create \
--account-name $STORAGE \
--account-key $ACCOUNT_KEY \
--name private-docs \
--public-access off

# List containers
az storage container list \
--account-name $STORAGE \
--account-key $ACCOUNT_KEY \
--output table
```

Q2

Upload a file to both containers. Get the public URL for the image, and generate a SAS token for the private document valid for 2 hours.

Medium

Scenario: Demonstrate secure file access patterns for different scenarios.

■ Azure CLI Solution

```
# Create test files
echo ' Hello World!' > sample-image.html
echo 'CONFIDENTIAL: Q3 Financial Report' > private-report.txt

# Upload to public container
az storage blob upload \
--account-name $STORAGE \
--account-key $ACCOUNT_KEY \
--container-name public-images \
--name sample-image.html \
--file sample-image.html

# Get public URL (no auth needed)
echo "Public URL: https://${STORAGE}.blob.core.windows.net/public-images/sample-image.html"

# Upload to private container
az storage blob upload \
--account-name $STORAGE \
--account-key $ACCOUNT_KEY \
--container-name private-docs \
--name private-report.txt \
--file private-report.txt

# Generate SAS token valid for 2 hours
EXPIRY=$(date -u -d '2 hours' '+%Y-%m-%dT%H:%MZ' 2>/dev/null || \
date -u -v+2H '+%Y-%m-%dT%H:%MZ')
SAS=$(az storage blob generate-sas \
--account-name $STORAGE \
--account-key $ACCOUNT_KEY \
--container-name private-docs \
```

```
--name private-report.txt \
--permissions r \
--expiry $EXPIRY \
--output tsv)
echo "SAS URL: https://${STORAGE}.blob.core.windows.net/private-docs/private-report.txt?${SAS}"
# This URL works for 2 hours, then expires automatically!
```

■ **Explanation:** SAS (Shared Access Signature) tokens grant time-limited, scoped access to blobs without sharing your account key. Use --permissions r (read), w (write), d (delete), l (list).

Q3

Enable static website hosting on your storage account and deploy a simple HTML page as the index.

Medium

Scenario: Host a static website (HTML/CSS/JS) for just pennies a month!

■ Azure CLI Solution

```
# Enable static website
az storage blob service-properties update \
--account-name $STORAGE \
--account-key $ACCOUNT_KEY \
--static-website \
--index-document index.html \
--404-document 404.html

# Create index.html
cat > index.html << 'EOF'
<!DOCTYPE html>
<html lang='en'>
<head><meta charset='UTF-8'><title>Azure Static Site</title></head>
<body style='font-family:Arial;text-align:center;padding:60px'>
<h1>My Azure Static Website</h1>
<p>Hosted on Azure Blob Storage!</p>
<p>Cost: ~$0.02/GB/month</p>
</body></html>
EOF

# Upload to the special $web container
az storage blob upload \
--account-name $STORAGE \
--account-key $ACCOUNT_KEY \
--container-name '$web' \
--name index.html \
--file index.html \
--content-type 'text/html'

# Get the static website URL
az storage account show \
--name $STORAGE \
--resource-group rg-myapp-dev \
--query 'primaryEndpoints.web' \
--output tsv
```

LAB 8

Azure SQL Server & Database

■ 50 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create an Azure SQL Server with admin credentials
- Create and configure a SQL Database
- Connect with Azure Data Studio
- Run SQL queries and create tables
- Configure firewall rules and private endpoint
- Set up geo-redundant backup

Q1

Create a SQL Server named 'sql-lab-[yourname]-001' and a database named 'db-myapp'. Set the compute to Basic tier (5 DTUs). Allow your current IP to connect.

Medium

Scenario: Provision a managed SQL database for your web application.

■ Portal Solution

1	Search SQL databases Search 'SQL databases' → '+ Create'.
2	Basics RG: rg-myapp-dev. Database name: db-myapp. Server: Create new → Name: sql-lab-yourname-001, Region: East US, Auth: SQL auth, Admin: sqladmin, Password: Lab@Pass123!
3	Compute + storage Click 'Configure database' → Basic (5 DTUs, 2GB storage). Click Apply.
4	Networking Connectivity: Public endpoint. Add current client IP: YES. Allow Azure services: YES.
5	Review + Create Review and Create. Takes ~3 minutes.
6	Connect After deploy, go to SQL Server → Query editor (preview). Login: sqladmin / Lab@Pass123! — run: SELECT @@VERSION

■ Azure CLI Solution

```
YOUR='yourname001' # change this
# Create SQL Server
az sql server create \
--name sql-lab-$YOUR \
--resource-group rg-myapp-dev \
--location eastus \
--admin-user sqladmin \
--admin-password 'Lab@Pass123!'
```

```
# Allow your current IP
MY_IP=$(curl -s https://api.ipify.org)
echo "Your IP: $MY_IP"
az sql server firewall-rule create \
--resource-group rg-myapp-dev \
--server sql-lab-$YOUR \
--name AllowMyIP \
--start-ip-address $MY_IP \
--end-ip-address $MY_IP
# Allow Azure services (for App Service connections)
az sql server firewall-rule create \
--resource-group rg-myapp-dev \
--server sql-lab-$YOUR \
--name AllowAzureServices \
--start-ip-address 0.0.0.0 \
--end-ip-address 0.0.0.0
# Create the database
az sql db create \
--resource-group rg-myapp-dev \
--server sql-lab-$YOUR \
--name db-myapp \
--edition Basic \
--capacity 5 \
--max-size 2GB
# Get the connection string
az sql db show-connection-string \
--server sql-lab-$YOUR \
--name db-myapp \
--client ado.net
```

Q2

Connect to db-myapp using Azure Data Studio (or the portal Query Editor), create a 'Users' table, insert 3 records, and query them.

Medium

Scenario: Test your database with real SQL queries.

■ Azure CLI Solution

```
# Connect via sqlcmd (install if needed: sudo apt install sqlcmd)
SERVER='sql-lab-$YOUR.database.windows.net'
sqlcmd \
-S $SERVER \
-d db-myapp \
-U sqladmin \
-P 'Lab@Pass123!' \
-Q "
-- Create Users table
CREATE TABLE Users (
UserID INT IDENTITY(1,1) PRIMARY KEY,
Username NVARCHAR(50) NOT NULL,
Email NVARCHAR(100) NOT NULL,
CreatedAt DATETIME DEFAULT GETDATE()
);
-- Insert sample data
INSERT INTO Users (Username, Email) VALUES
('alice', 'alice@example.com'),
('bob', 'bob@example.com'),
```

```
( 'charlie', 'charlie@example.com' );
-- Query all users
SELECT * FROM Users;
-- Count users
SELECT COUNT(*) AS TotalUsers FROM Users;
"
# You can also run SQL via Azure CLI
az sql db execute \
--resource-group rg-myapp-dev \
--server sql-lab-$YOUR \
--database db-myapp \
--query-file ./my-script.sql
```

■ **TIP** Use Azure Data Studio (free download from Microsoft) for a full SQL IDE with IntelliSense, query results grid, and connection management.

Q3

List all databases on the SQL Server, check the database size, and then delete the database and server to save costs.

Easy

Scenario: Clean up all SQL resources after the lab.

■ Azure CLI Solution

```
# List all databases on the server
az sql db list \
--resource-group rg-myapp-dev \
--server sql-lab-$YOUR \
--output table
# Show database details (size, tier, DTUs)
az sql db show \
--resource-group rg-myapp-dev \
--server sql-lab-$YOUR \
--name db-myapp \
--query '{Name:name, Edition:sku.tier, MaxSizeBytes:maxSizeBytes}' \
--output table
# Delete the database
az sql db delete \
--resource-group rg-myapp-dev \
--server sql-lab-$YOUR \
--name db-myapp \
--yes
# Delete the server (also deletes all remaining DBs)
az sql server delete \
--name sql-lab-$YOUR \
--resource-group rg-myapp-dev \
--yes
```


PART 5

Identity & Security

Entra ID, RBAC, Service Principals & Key Vault

Lab 9: Azure Entra ID

Lab 10: Azure RBAC

Lab 11: Service Principals & Key Vault

LAB 9

Azure Entra ID (Azure Active Directory)

■ 40 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create users and groups in Entra ID
- Assign users to groups
- Configure guest user access
- Enable MFA for specific users
- Set up Conditional Access policies
- Review sign-in logs

Q1

Create two users (Alice Developer and Bob Viewer), two groups (sg-developers and sg-viewers), and add users to appropriate groups.

Medium

Scenario: Set up Entra ID structure for a new project team.

■ Portal Solution

1

Open Entra ID

Search 'Microsoft Entra ID'. In left menu click 'Users'.

2

Create Alice

Click '+ New user' → Create new user. UPN: alice@yourtenant.onmicrosoft.com. Display name: Alice Developer. Auto-generate password (note it down). Click Create.

3

Create Bob

Same: UPN bob@yourtenant.onmicrosoft.com. Display name: Bob Viewer.

4

Create groups

Left menu → 'Groups' → '+ New group'. Type: Security. Name: sg-developers. No owners/members yet. Click Create. Repeat for sg-viewers.

5

Add Alice to sg-developers

Click sg-developers → Members → '+ Add members' → search Alice → Select.

6

Add Bob to sg-viewers

Click sg-viewers → Members → '+ Add members' → search Bob → Select.

■ Azure CLI Solution

```
# Get your tenant domain
DOMAIN=$(az account show --query 'tenantId' -o tsv)
TENANT_DOMAIN=$(az rest --method GET \
--url 'https://graph.microsoft.com/v1.0/domains' \
--query 'value[0].id' -o tsv 2>/dev/null || echo 'yourtenant.onmicrosoft.com')
echo "Tenant domain: $TENANT_DOMAIN"

# Create Alice (Developer)
az ad user create \
--display-name 'Alice Developer' \
```

```
--user-principal-name alice@$TENANT_DOMAIN \
--password 'TempP@ss123!' \
--force-change-password-next-sign-in true
ALICE_ID=$(az ad user show \
--id alice@$TENANT_DOMAIN \
--query id -o tsv)
# Create Bob (Viewer)
az ad user create \
--display-name 'Bob Viewer' \
--user-principal-name bob@$TENANT_DOMAIN \
--password 'TempP@ss123!' \
--force-change-password-next-sign-in true
BOB_ID=$(az ad user show \
--id bob@$TENANT_DOMAIN --query id -o tsv)
# Create groups
az ad group create \
--display-name 'sg-developers' \
--mail-nickname 'sg-developers'
az ad group create \
--display-name 'sg-viewers' \
--mail-nickname 'sg-viewers'
DEV_GRP=$(az ad group show --group sg-developers --query id -o tsv)
VIEW_GRP=$(az ad group show --group sg-viewers --query id -o tsv)
# Add members to groups
az ad group member add --group $DEV_GRP --member-id $ALICE_ID
az ad group member add --group $VIEW_GRP --member-id $BOB_ID
# Verify group membership
az ad group member list --group sg-developers --output table
az ad group member list --group sg-viewers --output table
```

Q2

List all users and groups in your tenant, find users who have not signed in for more than 30 days, and delete a test user.

Medium

Scenario: Regular user auditing is a security best practice.

■ Azure CLI Solution

```
# List all users
az ad user list --output table
# List with specific fields
az ad user list \
--query '[].{DisplayName:displayName, UPN:userPrincipalName, AccountEnabled:accountEnabled}' \
--output table
# List all groups
az ad group list --output table
# Find users who haven't signed in (requires Graph query)
az rest \
--method GET \
--url 'https://graph.microsoft.com/v1.0/users?$select=displayName,signInActivity,userPrincipalName'
# Disable a user account (safer than deleting)
az ad user update \
--id bob@$TENANT_DOMAIN \
--account-enabled false
# Delete a user permanently
az ad user delete --id alice@$TENANT_DOMAIN
# List recently deleted users (soft-deleted, 30 day restore window)
```

```
az rest \  
--method GET \  
--url 'https://graph.microsoft.com/v1.0/directory/deletedItems/microsoft.graph.user'
```

LAB 10

Azure RBAC

■ 40 min

Difficulty: ★★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Assign built-in roles to users and groups
- Create a custom role with specific permissions
- Verify role assignments work correctly
- Use PIM (Privileged Identity Management) concepts
- Audit all role assignments in a subscription

Q1

**Assign the 'Reader' role to Bob Viewer on resource group 'rg-myapp-dev'.
Assign 'Contributor' to the sg-developers group on the same resource group.
Verify both assignments.**

Medium

Scenario: Implement least-privilege access for your team.

■ Portal Solution

1

Navigate to RG

Go to rg-myapp-dev → Access control (IAM).

2

Add Reader for Bob

Click '+ Add' → 'Add role assignment'. Role: Reader. Members: Select 'User' → search Bob Viewer → Select. Click 'Review + assign' twice.

3

Add Contributor for group

Repeat: Role: Contributor. Members: Select 'Group' → search sg-developers → Select. Click 'Review + assign'.

4

Verify assignments

Click 'Role assignments' tab. Filter by scope. You should see both assignments listed.

5

Test as Bob

Open incognito browser. Login as bob@yourtenant.onmicrosoft.com. Navigate to portal. Try to create a resource — it should be denied!

■ Azure CLI Solution

```
# Assign Reader role to Bob on the resource group
az role assignment create \
  --assignee bob@$TENANT_DOMAIN \
  --role 'Reader' \
  --resource-group rg-myapp-dev
# Assign Contributor role to sg-developers group
DEV_GRP_ID=$(az ad group show --group sg-developers --query id -o tsv)
az role assignment create \
  --assignee-object-id $DEV_GRP_ID \
  --assignee-principal-type Group \
  --role 'Contributor' \
```

```
--resource-group rg-myapp-dev
# List all role assignments in the RG
az role assignment list \
--resource-group rg-myapp-dev \
--output table
# Filter: show only assignments for Bob
az role assignment list \
--resource-group rg-myapp-dev \
--assignee bob@$TENANT_DOMAIN \
--output table
# Remove Bob's Reader role
az role assignment delete \
--assignee bob@$TENANT_DOMAIN \
--role 'Reader' \
--resource-group rg-myapp-dev
```

Q2

Create a custom RBAC role called 'VM Operator' that can Start, Stop, and Restart VMs but cannot create or delete them, and cannot read secrets.

Hard

Scenario: Your operations team needs to manage VM power states but not create infrastructure.

Azure CLI Solution

```
# Create the custom role definition JSON
cat > vm-operator-role.json << 'EOF'
{
  "Name": "VM Operator",
  "Description": "Can start, stop, restart VMs. Cannot create, delete, or read secrets.",
  "Actions": [
    "Microsoft.Compute/virtualMachines/start/action",
    "Microsoft.Compute/virtualMachines/powerOff/action",
    "Microsoft.Compute/virtualMachines/restart/action",
    "Microsoft.Compute/virtualMachines/read",
    "Microsoft.Compute/virtualMachines/instanceView/read",
    "Microsoft.Resources/subscriptions/resourceGroups/read"
  ],
  "NotActions": [
    "Microsoft.KeyVault/*/read"
  ],
  "DataActions": [],
  "NotDataActions": [],
  "AssignableScopes": [
    "/subscriptions/<YOUR_SUBSCRIPTION_ID>"
  ]
}
EOF
# Get subscription ID
SUB_ID=$(az account show --query id -o tsv)
sed -i "s/<YOUR_SUBSCRIPTION_ID>/$SUB_ID/" vm-operator-role.json
# Create the custom role
az role definition create --role-definition vm-operator-role.json
# List custom roles
az role definition list --custom-role-only true --output table
# Assign the custom role to a user
az role assignment create \
--assignee bob@$TENANT_DOMAIN \
```

```
--role 'VM Operator' \  
--resource-group rg-myapp-dev  
# Delete the custom role (after removing assignments)  
az role definition delete --name 'VM Operator'
```

■ *Explanation:* Custom roles must be in AssignableScopes which defines where the role can be assigned. You can use subscription, management group, or specific resource group scope.

LAB 11

Service Principals & Key Vault

■ 50 min

Difficulty: ★★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create a service principal for automation
- Create a Key Vault and store secrets
- Grant the service principal access to Key Vault
- Retrieve secrets programmatically
- Rotate secrets and revoke access
- Use Managed Identity (no password needed!)

Q1

Create a Service Principal named 'sp-lab-automation' scoped to rg-myapp-dev with Contributor role. Save its credentials securely.

Medium

Scenario: Your CI/CD pipeline needs a service account to deploy resources.

■ Portal Solution

1

Register App

Search 'App registrations' in Entra ID → '+ New registration'. Name: sp-lab-automation. Supported account types: Accounts in this directory only. Click Register.

2

Create client secret

Go to the app → Certificates & secrets → '+ New client secret'. Description: lab-secret. Expires: 1 year. Click Add. COPY THE VALUE IMMEDIATELY.

3

Get App/Client ID

On Overview page, copy: Application (client) ID and Directory (tenant) ID.

4

Assign role

Go to rg-myapp-dev → Access control (IAM) → '+ Add' → Add role assignment. Role: Contributor. Members: search for sp-lab-automation. Assign.

■ Azure CLI Solution

```
# Get subscription ID
SUB_ID=$(az account show --query id -o tsv)

# Create SP with Contributor on the resource group
az ad sp create-for-rbac \
  --name sp-lab-automation \
  --role Contributor \
  --scopes /subscriptions/$SUB_ID/resourceGroups/rg-myapp-dev \
  --output json

# Output (SAVE THIS SECURELY – shown only once!):
# {
#   'appId': '<CLIENT_ID>', <- Service Principal ID
#   'displayName': 'sp-lab-automation',
#   'password': '<CLIENT_SECRET>', <- Save in Key Vault!
#   'tenant': '<TENANT_ID>'
```

```
# }
# Test: login as the service principal
az login \
--service-principal \
--username <CLIENT_ID> \
--password <CLIENT_SECRET> \
--tenant <TENANT_ID>
# List SPs
az ad sp list --display-name sp-lab-automation --output table
# Delete SP
az ad sp delete --id <CLIENT_ID>
```

Q2

Create a Key Vault, store the database password as a secret named 'DatabasePassword', and grant the SP access to read secrets only.

Hard

Scenario: Centralise all secrets in Key Vault — no secrets in code or environment variables!

■ Portal Solution

- 1 **Create Key Vault**
Search 'Key vaults' → '+ Create'. RG: rg-myapp-dev. Name: kv-lab-yourname-001 (globally unique). Region: East US. Pricing: Standard.
- 2 **Access policy**
Permission model: Vault access policy (classic). Keep your account as Key Vault Administrator.
- 3 **Create**
Review + Create.
- 4 **Add secret**
Go to Key Vault → Secrets → '+ Generate/Import'. Upload options: Manual. Name: DatabasePassword. Value: Lab@SecretPass123!. Click Create.
- 5 **Grant SP access**
Go to Key Vault → Access policies → '+ Create'. Permissions: Secret permissions: Get, List. Principal: search for sp-lab-automation. Click Create.

■ Azure CLI Solution

```
# Create Key Vault
KV_NAME="kv-lab-$(echo $USER | head -c4)-001"
echo "Key Vault name: $KV_NAME"
az keyvault create \
--name $KV_NAME \
--resource-group rg-myapp-dev \
--location eastus \
--sku standard
# Store secrets
az keyvault secret set \
--vault-name $KV_NAME \
--name 'DatabasePassword' \
--value 'Lab@SecretPass123!'
az keyvault secret set \
--vault-name $KV_NAME \
--name 'ApiKey' \
--value 'sk-abc123xyzlabkey'
```

```
# Get the SP's object ID
SP_ID=$(az ad sp show \
--id sp-lab-automation \
--query id -o tsv)
# Grant SP read-only access to secrets
az keyvault set-policy \
--name $KV_NAME \
--object-id $SP_ID \
--secret-permissions get list
# Read a secret (as yourself)
az keyvault secret show \
--vault-name $KV_NAME \
--name 'DatabasePassword' \
--query value -o tsv
# List all secrets
az keyvault secret list \
--vault-name $KV_NAME \
--output table
# Show secret versions (history)
az keyvault secret list-versions \
--vault-name $KV_NAME \
--name 'DatabasePassword'
# Delete and purge a secret
az keyvault secret delete --vault-name $KV_NAME --name ApiKey
az keyvault secret purge --vault-name $KV_NAME --name ApiKey
```

■ *Explanation:* Key Vault supports soft-delete by default — deleted secrets are recoverable for 90 days. Use 'purge' to permanently remove. Always enable Key Vault diagnostic logs to audit who accessed what.

PART 6

Monitoring

Application Insights & Azure Monitor

Lab 12: Application Insights

Lab 13: Azure Monitor & Alerts

LAB 12

Application Insights

■ 40 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create Application Insights and Log Analytics Workspace
- Connect App Insights to an App Service
- Run KQL queries on telemetry data
- Create availability tests
- Set up failure alerts
- Analyse performance with Application Map

Q1

Create an Application Insights resource 'appi-lab-prod' with a Log Analytics workspace 'law-lab-prod', then connect it to your App Service.

Medium

Scenario: Enable monitoring for your production web application.

■ Portal Solution

1

Create Log Analytics

Search 'Log Analytics workspaces' → '+ Create'. RG: rg-myapp-dev. Name: law-lab-prod. Region: East US. Pricing: Pay-as-you-go.

2

Create App Insights

Search 'Application Insights' → '+ Create'. RG: rg-myapp-dev. Name: appi-lab-prod. Region: East US. Resource mode: Workspace-based. Select law-lab-prod workspace.

3

Get connection string

Go to appi-lab-prod → Overview → copy 'Connection String'.

4

Connect to App Service

Go to your App Service → Application Insights → Turn on → select appi-lab-prod → Apply.

5

Verify

After 5 minutes, go to appi-lab-prod → Live Metrics. Make requests to your app and watch them appear!

■ Azure CLI Solution

```
# Create Log Analytics Workspace
az monitor log-analytics workspace create \
--resource-group rg-myapp-dev \
--workspace-name law-lab-prod \
--location eastus
# Get workspace ID
WS_ID=$(az monitor log-analytics workspace show \
--resource-group rg-myapp-dev \
--workspace-name law-lab-prod \
--query id -o tsv)
# Create Application Insights
```

```
az monitor app-insights component create \
--app appi-lab-prod \
--resource-group rg-myapp-dev \
--location eastus \
--workspace $WS_ID
# Get the connection string
CONN_STR=$(az monitor app-insights component show \
--app appi-lab-prod \
--resource-group rg-myapp-dev \
--query connectionString -o tsv)
echo "Connection String: $CONN_STR"
# Connect to App Service
az webapp config appsettings set \
--resource-group rg-myapp-dev \
--name webapp-lab-$YOUR_NAME \
--settings \
APPLICATIONINSIGHTS_CONNECTION_STRING="$CONN_STR" \
ApplicationInsightsAgent_EXTENSION_VERSION=~3 \
XDT_MicrosoftApplicationInsights_Mode='recommended'
```

Q2

Write and run 5 useful KQL queries in Log Analytics to analyse your application's telemetry.

Hard

Scenario: KQL is the querying language for all Azure monitoring data.

■ Azure CLI Solution

```
# ■■■ Run these in Application Insights → Logs ■■■
# Query 1: Request count by response code (last 24h)
requests
| where timestamp > ago(24h)
| summarize RequestCount = count() by resultCode
| order by RequestCount desc
| render barchart

# Query 2: Average response time per endpoint
requests
| where timestamp > ago(1h)
| summarize AvgDuration = avg(duration), P95 = percentile(duration, 95) by name
| order by AvgDuration desc

# Query 3: Exceptions with full details
exceptions
| where timestamp > ago(24h)
| project timestamp, type, outerMessage, severityLevel, operation_Name
| order by timestamp desc
| take 20

# Query 4: Daily active users (unique sessions)
requests
| where timestamp > ago(7d)
| summarize UniqueUsers = dcount(user_Id) by bin(timestamp, 1d)
| render timechart

# Query 5: Failed requests with error details
requests
| where timestamp > ago(1h)
| where success == false
| join kind=leftouter exceptions on operation_Id
| project timestamp, name, resultCode, outerMessage, duration
```

| order by timestamp desc

Q3

Create an Availability Test that pings your web app every 5 minutes from 3 locations and alerts you if it's down.

Medium

Scenario: Proactively monitor if your website is up from multiple locations around the world.

■ Portal Solution

1	Open App Insights Navigate to appi-lab-prod.
2	Availability In left menu under Investigate, click 'Availability'.
3	Add test Click '+ Add Standard test'. Test name: ping-webapp. URL: https://webapp-lab-yourname.azurewebsites.net. Test frequency: 5 minutes. Test locations: select 3 (e.g., East US, West Europe, Southeast Asia).
4	Configure alerts Enable 'Enable alerts'. Alert threshold: 2 failed locations. Notification: your email. Click 'Create'.
5	Verify After 15 minutes, the availability tab shows a map with green checkmarks for each test location.

■ Azure CLI Solution

```
# Create availability test via CLI
az monitor app-insights web-test create \
--resource-group rg-myapp-dev \
--app-name appi-lab-prod \
--name 'ping-webapp' \
--location eastus \
--defined-web-test-type standard \
--http-verb GET \
--expected-status-code 200 \
--frequency 300 \
--locations '["Id":"us-ca-sjc-azr"],["Id":"emea-gb-db3-azr"],["Id":"apac-sg-sin-azr"]' \
--url https://webapp-lab-yourname.azurewebsites.net
```

LAB 13

Azure Monitor & Alerts

■ 45 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create metric-based alerts for VMs and App Services
- Configure Action Groups for email/SMS notifications
- Enable diagnostic settings to send logs to Log Analytics
- Query VM performance data with KQL
- Create Log-based alerts
- Build a Monitor Workbook

Q1

Create an alert rule that sends an email when your App Service's HTTP 5xx error rate exceeds 5 errors in 5 minutes.

Medium

Scenario: Be alerted immediately when your app is throwing server errors in production.

■ Portal Solution

1

Go to Monitor

Search 'Monitor' → Alerts → '+ Create' → Alert rule.

2

Select scope

Click 'Select scope' → find your App Service → click it → Done.

3

Condition

Click 'Add condition' → search 'Http 5xx' → select it. Threshold: Static. Operator: Greater than. Value: 5. Aggregation: Count. Period: 5 minutes.

4

Action Group

Click 'Create action group'. Basics: Name: ag-webapp-alerts. Short name: webapp. Action: Email/SMS → email → enter your email. Click 'OK'.

5

Alert details

Name: alert-http5xx. Severity: 1 (Error). Enable: Yes.

6

Create

Click 'Review + create' → 'Create'.

■ Azure CLI Solution

```
# Step 1: Create an Action Group
az monitor action-group create \
--resource-group rg-myapp-dev \
--name ag-webapp-alerts \
--short-name webapp \
--action email youremail 'your@email.com'

# Get Action Group resource ID
AG_ID=$(az monitor action-group show \
--resource-group rg-myapp-dev \
```

```
--name ag-webapp-alerts \
--query id -o tsv)
# Get App Service resource ID
APP_ID=$(az webapp show \
--resource-group rg-myapp-dev \
--name webapp-lab-$YOUR_NAME \
--query id -o tsv)
# Create the metric alert
az monitor metrics alert create \
--name alert-http5xx \
--resource-group rg-myapp-dev \
--scopes $APP_ID \
--condition "count Http5xx > 5" \
--window-size 5m \
--evaluation-frequency 1m \
--severity 1 \
--action $AG_ID \
--description 'Alert when app throws more than 5 HTTP 5xx errors in 5 minutes'
# List all alerts
az monitor metrics alert list \
--resource-group rg-myapp-dev \
--output table
# Delete alert
az monitor metrics alert delete \
--resource-group rg-myapp-dev \
--name alert-http5xx
```

Q2

Enable diagnostic settings on your App Service to send all logs and metrics to Log Analytics workspace 'law-lab-prod'.

Medium

Scenario: Centralise all logs in Log Analytics for unified querying and alerting.

■ Portal Solution

1	Go to App Service Navigate to webapp-lab-yourname → Diagnostic settings.
2	Add diagnostic setting Click '+ Add diagnostic setting'. Name: diag-webapp-prod.
3	Select logs Check: AppServiceHTTPLogs, AppServiceConsoleLogs, AppServiceAppLogs, AppServiceAuditLogs. Also check 'AllMetrics'.
4	Destination Check 'Send to Log Analytics workspace'. Select law-lab-prod.
5	Save Click Save. After 5-10 minutes, logs flow to Log Analytics.

■ Azure CLI Solution

```
# Get resource IDs
APP_ID=$(az webapp show -g rg-myapp-dev -n webapp-lab-$YOUR_NAME --query id -o tsv)
WS_ID=$(az monitor log-analytics workspace show \
-g rg-myapp-dev -n law-lab-prod --query id -o tsv)
```

```
# Enable diagnostic settings
az monitor diagnostic-settings create \
--name diag-webapp-prod \
--resource $APP_ID \
--workspace $WS_ID \
--logs '[
{"category":"AppServiceHTTPLogs","enabled":true},
{"category":"AppServiceConsoleLogs","enabled":true},
{"category":"AppServiceAppLogs","enabled":true},
{"category":"AppServiceAuditLogs","enabled":true}
]' \
--metrics '["category":"AllMetrics","enabled":true]'
```

Query the logs in Log Analytics (after 10 mins)

```
az monitor log-analytics query \
--workspace $WS_ID \
--analytics-query \
'AppServiceHTTPLogs | take 20 | project TimeGenerated, CsMethod, CsUriStem, ScStatus'
```


PART 7

DevOps & Pipelines

Azure Repos, Boards & YAML Pipelines

Lab 14: Azure Repos & Boards

Lab 15: Azure YAML Pipelines

LAB 14

Azure Repos & Boards

■ 45 min

Difficulty: ★★

■ Portal + ■ Azure CLI

■ Learning Objectives

- Create an Azure DevOps project
- Push code to Azure Repos
- Create Pull Requests with branch policies
- Create Epics, Features, Stories, and Tasks in Boards
- Link work items to commits and PRs
- Run a Sprint and update a Kanban board

Q1

Create a new Azure DevOps organization and project. Initialize a Git repository and push a sample Node.js app to Azure Repos.

Medium

Scenario: Set up source control for your project team.

■ Portal Solution

1

Create DevOps org

Go to dev.azure.com → 'New organization' → name it (yourname-devops) → Create.

2

Create project

Click '+ New project'. Name: MyAwesomeApp. Visibility: Private. Version control: Git. Work item process: Agile. Click Create.

3

Explore Repos

Click 'Repos' in the left menu. Click 'Initialize' to create an empty repo with README.

4

Clone URL

Copy the HTTPS clone URL from Repos → Files → Clone.

■ Azure CLI Solution

```
# Install Azure DevOps CLI extension
az extension add --name azure-devops

# Configure defaults
az devops configure \
  --defaults organization=https://dev.azure.com/yourorg project=MyAwesomeApp

# Create project via CLI
az devops project create \
  --name MyAwesomeApp \
  --organization https://dev.azure.com/yourorg \
  --visibility private \
  --source-control git \
  --process agile

# Get repo clone URL
az repos show \
  --repository MyAwesomeApp \
  --query remoteUrl -o tsv
```

```
# Clone the repo
git clone https://dev.azure.com/yourorg/MyAwesomeApp/_git/MyAwesomeApp
cd MyAwesomeApp

# Create a Node.js app
cat > index.js << 'EOF'
const express = require('express');
const app = express();
app.get('/', (req, res) => res.send('Hello from Azure DevOps!'));
app.listen(process.env.PORT || 3000);
EOF

echo '{"name":"myapp","scripts":{"start":"node index.js"},"dependencies":{"express":"^4.18.0"}}' >
package.json

# Commit and push
git add .
git commit -m 'feat: initial Node.js app'
git push origin main
```

Q2

Create a feature branch, make a code change, push it, and create a Pull Request with a reviewer assigned. Set up a branch policy requiring 1 reviewer and a successful build.

Hard

Scenario: Enforce code review and CI gates before merging to main.

■ Azure CLI Solution

```
# Create a feature branch
git checkout -b feature/add-health-endpoint

# Add a new endpoint to index.js
cat >> index.js << 'EOF'
app.get('/health', (req, res) => res.json({status: 'healthy', timestamp: new Date()}));
EOF

git add index.js
git commit -m 'feat: add /health endpoint for monitoring'
git push origin feature/add-health-endpoint

# Create a Pull Request via CLI
az repos pr create \
--repository MyAwesomeApp \
--source-branch feature/add-health-endpoint \
--target-branch main \
--title 'feat: add /health endpoint' \
--description 'Adds a /health endpoint for uptime monitoring and load balancers.' \
--reviewers 'alice@yourtenant.onmicrosoft.com'

# List open PRs
az repos pr list --output table

# Set branch policy: require 1 approver on main
az repos policy approver-count create \
--repository-id <REPO_ID> \
--branch main \
--is-enabled true \
--is-blocking true \
--minimum-approver-count 1 \
--creator-vote-counts false

# Approve and complete PR
PR_ID=$(az repos pr list --query '[0].pullRequestId' -o tsv)
az repos pr update --id $PR_ID --status completed --merge-strategy squash
```

Q3

Create an Epic, 2 Features, and 4 User Stories in Azure Boards. Move one story to In Progress and link it to a commit.

Medium

Scenario: Manage your project work items using Agile methodology.

■ Azure CLI Solution

```
# Create an Epic
az boards work-item create \
--title 'User Authentication System' \
--type Epic \
--assigned-to alice@yourtenant.onmicrosoft.com
EPIC_ID=$(az boards work-item create \
--title 'User Authentication System' --type Epic \
--query id -o tsv)

# Create Features
F1=$(az boards work-item create \
--title 'Login Page' --type Feature --query id -o tsv)
F2=$(az boards work-item create \
--title 'Password Reset Flow' --type Feature --query id -o tsv)

# Create User Stories
S1=$(az boards work-item create \
--title 'As a user, I can log in with email and password' \
--type 'User Story' --query id -o tsv)
S2=$(az boards work-item create \
--title 'As a user, I can see an error if my password is wrong' \
--type 'User Story' --query id -o tsv)

# Move Story 1 to Active (In Progress)
az boards work-item update --id $S1 --state Active

# List work items
az boards work-item show --id $S1

# Query work items
az boards query \
--wiql "SELECT [Id],[Title],[State] FROM WorkItems WHERE [State]='Active'"

```

Azure YAML Pipelines

■ Portal + ■ Azure CLI

```
include:  
- main # Run pipeline on PRs targeting main  
  
variables:  
APP_NAME: 'webapp-lab-${Build.SourceBranchName}'  
RG: 'rg-myapp-dev'  
NODE_VERSION: '18.x'  
buildConfiguration: 'Release'  
  
stages:  
# ■■ STAGE 1: Build & Test ████████████████████████████████████████  
- stage: Build  
displayName: '■ Build & Test'  
jobs:  
- job: BuildAndTest  
displayName: 'Install, Lint, Test'  
pool:  
vmImage: ubuntu-latest  
steps:  
- task: NodeTool@0  
displayName: 'Use Node.js ${NODE_VERSION}'  
inputs:  
versionSpec: '${NODE_VERSION}'  
- script: |  
npm ci  
displayName: '■ Install dependencies'  
- script: |  
npm run lint 2>/dev/null || echo 'No lint script, skipping'  
displayName: '■ Run linter'  
- script: |  
npm test 2>/dev/null || echo 'No tests yet'  
displayName: '■ Run tests'  
- task: PublishTestResults@2  
condition: always()  
inputs:  
testResultsFormat: 'JUnit'  
testResultsFiles: '**/test-results.xml'  
failTaskOnFailedTests: false  
# ■■ STAGE 2: Package ████████████████████████████████████████  
- stage: Package  
displayName: '■ Create Artifact'  
dependsOn: Build  
condition: succeeded()  
jobs:  
- job: PackageApp  
pool:  
vmImage: ubuntu-latest  
steps:  
- task: NodeTool@0  
inputs:  
versionSpec: '${NODE_VERSION}'  
- script: npm ci --production  
displayName: 'Install prod dependencies only'  
- task: ArchiveFiles@2  
displayName: '■ Zip application files'  
inputs:  
rootFolderOrFile: '${System.DefaultWorkingDirectory}'
```

```
includeRootFolder: false  
archiveType: zip  
archiveFile: '$(Build.ArtifactStagingDirectory)/app.zip'  
replaceExistingArchive: true  
  
- task: PublishPipelineArtifact@1  
displayName: '■ Publish artifact'  
inputs:  
targetPath: '$(Build.ArtifactStagingDirectory)'  
artifact: 'webapp-drop'  
  
# ■■■ STAGE 3: Deploy to Dev ■■■  
  
- stage: DeployDev  
displayName: '■ Deploy to Dev'  
dependsOn: Package  
condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))  
jobs:  
- deployment: DeployWebApp  
displayName: 'Deploy to App Service'  
environment: 'development'  
pool:  
vmImage: ubuntu-latest  
strategy:  
runOnce:  
deploy:  
steps:  
- task: DownloadPipelineArtifact@2  
inputs:  
artifact: 'webapp-drop'  
path: '$(Pipeline.Workspace)/drop'  
- task: AzureWebApp@1  
displayName: '■■■ Deploy to Azure App Service'  
inputs:  
azureSubscription: 'sc-azure-lab' # Your service connection  
appType: webAppLinux  
appName: '$(APP_NAME)'  
package: '$(Pipeline.Workspace)/drop/app.zip'  
runtimeStack: 'NODE|18-lts'  
- task: AzureCLI@2  
displayName: '■ Smoke test deployment'  
inputs:  
azureSubscription: 'sc-azure-lab'  
scriptType: bash  
scriptLocation: inlineScript  
inlineScript: |  
URL=$(az webapp show -g $(RG) -n $(APP_NAME) --query defaultHostName -o tsv)  
STATUS=$(curl -s -o /dev/null -w '%{http_code}' https://$URL)  
echo "App URL: https://$URL"  
echo "HTTP Status: $STATUS"  
  
[ $STATUS -eq 200 ] && echo '■ App is healthy!' || (echo '■ App returned non-200!' && exit 1)
```

Hard

■ Portal Solution

1	Create variable group Azure DevOps → Pipelines → Library → '+ Variable group'. Name: lab-secrets.
2	Link to Key Vault Toggle 'Link secrets from an Azure key vault'. Select your service connection and Key Vault kv-lab-yourname-001. Authorize. Click '+ Add' to select specific secrets.
3	Reference in pipeline In your YAML, add 'group: lab-secrets' under variables. Then use: \$(DatabasePassword)

■ Azure CLI Solution

```
# Create variable group via CLI
az pipelines variable-group create \
--name lab-vars \
--variables APP_ENV=development NODE_ENV=production
# Add a secret variable (value is masked in logs)
az pipelines variable-group variable create \
--group-id <group-id> \
--name API_KEY \
--value 'super-secret-key' \
--secret true
# Use variable group in YAML:
# variables:
# - group: lab-vars # all vars from group
# - name: MY_VAR
# value: 'direct value'
# In steps, use as: $(API_KEY), $(APP_ENV)
# Secret variables are never printed in logs
# List variable groups
az pipelines variable-group list --output table
```

Q3

Add a manual approval gate between the Dev and Production stages so a human must approve before production deployment.

Hard

Scenario: Production deployments should require explicit human approval.

■ Portal Solution

1	Create environment Azure DevOps → Pipelines → Environments → '+ New environment'. Name: production. Click Create.
2	Add approval check Click on 'production' environment → Approvals and checks → '+' → Approvals. Add your email or a user. Instructions: 'Verify dev deployment before approving prod'. Timeout: 24 hours. Click Create.
3	Update YAML Add a 'DeployProd' stage depending on 'DeployDev' using environment: production. The pipeline will pause and wait for approval!

■ Azure CLI Solution

```
# Add to azure-pipelines.yml AFTER the DeployDev stage:
# ■ STAGE 4: Deploy to Production (requires approval) ■
- stage: DeployProd
```

```
displayName: '■ Deploy to PRODUCTION'
dependsOn: DeployDev
condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
jobs:
- deployment: DeployProdApp
  displayName: 'Deploy to Production App Service'
  environment: 'production' # <-- This triggers the approval gate!
  pool:
    vmImage: ubuntu-latest
  strategy:
    runOnce:
      deploy:
        steps:
        - task: AzureWebApp@1
  inputs:
    azureSubscription: 'sc-azure-lab'
    appType: webAppLinux
    appName: 'webapp-lab-prod'
    package: '$(Pipeline.Workspace)/webapp-drop/app.zip'
    # When pipeline reaches DeployProd, it PAUSES
    # An approver gets an email: 'Deployment waiting for your approval'
    # They click Approve → deployment continues
    # They click Reject → pipeline fails with message
```

■ *Explanation:* Environment approvals decouple the automation from the authorization. The pipeline is fully automated but can't proceed to production without a human decision. This is key for compliance and governance!

Complete CLI Quick Reference

■ Portal + ■ Azure CLI

All Labs — Command Reference

```

COMPLETE AZURE CLI DEPENDENCY
#
# AUTHENTICATION
#
az login
az account show
az account list --output table
az account set --subscription 'NAME_OR_ID'
#
# RESOURCE GROUPS
#
az group create --name RG --location eastus --tags Env=Dev
az group list --output table
az group show --name RG
az group delete --name RG --yes --no-wait
az lock create --name LOCK --resource-group RG --lock-type CanNotDelete
az lock delete --name LOCK --resource-group RG
#
# NETWORKING
#
az network vnet create -g RG -n VNET --address-prefix 10.0.0.0/16
az network vnet subnet create -g RG --vnet-name VNET -n SUBNET --address-prefix 10.0.1.0/24
az network nsg create -g RG -n NSG
az network nsg rule create -g RG --nsg-name NSG -n RULE --priority 100 --direction Inbound \
--source-address-prefixes Internet --destination-port-ranges 80 --protocol Tcp --access Allow
az network vnet subnet update -g RG --vnet-name VNET -n SUBNET --network-security-group NSG
az network vnet peering create -g RG -n PEER --vnet-name VNET1 --remote-vnet VNET2_ID
--allow-vnet-access
#
# VIRTUAL MACHINES
#
az vm create -g RG -n VM --image Ubuntu2204 --size Standard_B1s --admin-username azureuser
--generate-ssh-keys
az vm open-port -g RG -n VM --port 80
az vm list-ip-addresses -g RG -n VM --query '[0].virtualMachine.network.publicIpAddresses[0].ipAddress'
-o tsv
az vm stop/start/deallocate/restart/delete -g RG -n VM
az vm resize -g RG -n VM --size Standard_B2s
az vm auto-shutdown -g RG -n VM --time 2200 --email 'you@email.com'
az vm disk attach -g RG --vm-name VM -n DISK --size-gb 32 --sku Standard_LRS --new
#
# APP SERVICE
#
az appservice plan create -g RG -n PLAN --sku FREE --is-linux

```

[illegible]

```
# ████████████████████████████████████████████████████████████████  
# AZURE DEVOPS  
# ████████████████████████████████████████████████████████████████  
  
az extension add --name azure-devops  
  
az devops configure --defaults organization=URL project=PROJECT  
  
az repos pr create --repository REPO --source-branch BRANCH --target-branch main --title TITLE  
  
az boards work-item create --title TITLE --type 'User Story'  
  
az pipelines variable-group create --name GROUP --variables KEY=VALUE
```